Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –
Системное и прикладное программное обеспечение

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Вводный эксперимент:
сравнение способов обхода графа»

Вариант: Read vs Write, Cache, 10M, Seq

Выполнил:
студент группы P3310
Чжун Цзяцзюнь

Проверил:
преподаватель
Максимов Андрей Николаевич

2026 г.

1. Цель, вариант и гипотеза

Цель — оценить, как изменение данных влияет на время последовательного обхода графа размером 10 МиБ с использованием файлового кеша ОС. Рассматриваются две реализации: `read/lseek` и `mmap`. В каждой реализации сравниваются чтение и чтение с обновлением поля `value`.

Группа	Реализация	Нагрузка	Аргументы
A	read/lseek	Чтение	Без флагов
B	read/lseek	Чтение и обновление	`--write`
C	mmap	Чтение	Без флагов
D	mmap	Чтение и обновление	`--write`

Во всех четырёх группах основного эксперимента фиксированы **Cache, 10 МиБ и Seq**. Флаг `--no-cache` отсутствует; A/C соответствуют `(-,-)`, B/D — `(WR,-)`. Вариант Read vs Write означает изменение одного фактора внутри каждого API. Сравнение API дополнительно объясняет механизмы доступа.

Гипотеза, сохранённая до пилота: обновление увеличит среднее время нормированного обхода для обеих реализаций. В `read/lseek` оно добавляет позиционирование и запись через системные вызовы. В `mmap` изменяются страницы общего отображения, что требует дополнительной работы с памятью и страницами. В режиме Cache обязательный синхронный сброс каждой записи на физический носитель отсутствует.

Для каждого API отдельно обозначим $\Delta = \mu_{\text{Write}} - \mu_{\text{Read}}$. Проверяются $H_0 : \Delta \leq 0$ и $H_1 : \Delta > 0$. Принятый заранее критерий — положительная нижняя граница двустороннего 95% доверительного интервала **парной разности** по раундам.

Материалы работы: ivanstylish/xv6-riscv — course-code/lab/intro-exp.

2. Паспорт и подготовка окружения

Параметр	Зафиксированное значение
Среда	Ubuntu 22.04.5 LTS, WSL2 на Windows; x86_64
Ядро	`6.6.87.2-microsoft-standard-WSL2`
CPU хоста	Intel Core i7-13700H; 14 физических ядер и 20 логических процессоров по Windows CIM
Топология гостя	20 логических CPU; 10 ядер × 2 потока по lscpu. Это представление WSL, а не физическая топология
Закрепление	`taskset -c 2`; доступный гостевой логический CPU 2
Память гостя	7835,2 МиБ RAM; перед серией доступно около 7086 МиБ
Swap	2048 МиБ; top показывает около 0,3 МиБ использования до и после серии
Файл / страница	10 485 760 байт / 4096 байт
Файловая система	ext4, /dev/sdd; рабочие данные находятся в домашнем каталоге Ubuntu
Компилятор	GCC 11.4.0; `-O2 -Wall`
Частота CPU	cpufreq/governor недоступен в WSL; постоянная частота, Turbo и температура не подтверждены
Питание хоста	Подключённое питание, заряд 99%; пользовательская схема питания Windows

Источники — снимки `data/formal/meta_before.txt`, `meta_after.txt` и `plan/host-before-pilot.json`. Хостовый снимок сделан до пилота; снимки гостя — непосредственно до и после основной серии.

Файл расположен на ext4 внутри Ubuntu, а не в общей директории `/mnt/c` или `/mnt/d`. Программы последовательно запускаются на одном гостевом логическом CPU. По всем 120 строкам `cpu_check=ok`. Закрепление в госте не гарантирует неизменного физического ядра Windows.

Граф 10 МиБ занимает около 0,13% общей RAM гостя. Его размер соответствует варианту и позволяет полностью прогреть файл в page cache; размер файлового кеша не тождественен размеру аппаратных кешей CPU.

Перед основной серией load average гостя равен 0,42/0,23/0,10, после — 1,03/0,65/0,30; послесерийное значение может включать сам эксперимент. В начальном снимке гостевой CPU простаивает на 99,7%, но Windows CIM показывает общую загрузку хоста 18%.

Протокол кеша одинаков для каждой основной конфигурации: вне таймера восстанавливается исходный файл (`cp`), выполняется `sync`, затем полный `cat` в `/dev/null`. Это прогрев **файлового page cache**, а не гарантированное одинаковое состояние CPU cache или TLB. `sync` не отключает кеш и действует на файловые системы в целом.

3. Подготовка данных и ознакомительные запуски

Использованы исходные программы курса и штатный генератор. Сборка выполняется GCC с `-O2 -Wall`. Два графа имеют одинаковые размер 10 МиБ и seed 427:

```
python3 ../util/graphgen.py -s 10M --seed 427 --topology sequential --verify -o graph-seq.bin
python3 ../util/graphgen.py -s 10M --seed 427 --topology chain -b 0.5 --min-step-pages 2 --verify -o graph-rand.bin
```

У Seq физический порядок обхода совпадает с порядком записей. У Rand сохраняется форма цепи, но путь переставлен по физическим смещениям. Заголовок занимает 40 байт, запись — 24 байта; граф содержит **436 905 вершин**. Проверка подтверждает достижимость всех вершин и отсутствие циклов. В основной серии дополнительно проверены последовательные связи $i \to i + 1$ и отсутствие потомка у последней вершины.

3.1. Baseline: топология и переключатели

Ознакомительные запуски предусмотрены шагом 4.3 задания для read/lseek; этап 2 повторяет эти шаги для mmap. Поэтому baseline охватывает оба графа и все сочетания двух флагов. Основной эксперимент варианта при этом фиксирует **Seq, Cache и 10 МиБ**: статистические выборки собираются только для четырёх групп A–D. Значения baseline не входят ни в пилот, ни в основное N.

До планирования основной серии выполнены **16 функциональных запусков**: Seq/Rand × два API × Read/Write × Cache/NoCache, по одному обходу. Все завершились с кодом 0 и обработали 436 905 вершин. Перед каждым запуском выполнены восстановление, `sync` и `cat`; глобальный сброс кеша не применялся. Таблица содержит elapsed GNU time в секундах; её разрешение — 0,01 с.

API	Граф	Режим	Cache, с	NoCache, с
read/lseek	Seq	Чтение	0,09	25,68
read/lseek	Seq	Обновление	0,23	88,89
read/lseek	Rand	Чтение	0,11	41,97
read/lseek	Rand	Обновление	0,24	101,75
mmap	Seq	Чтение	0,00	0,00
mmap	Seq	Обновление	0,00	0,00
mmap	Rand	Чтение	0,01	0,01
mmap	Rand	Обновление	0,02	0,02

Read/Cache задаётся без флагов, Write/Cache — `--write`, Read/NoCache — `--no-cache`, Write/NoCache — оба флага. У read/lseek в Linux NoCache использует `O_DIRECT`. У mmap page cache остаётся: `madvise` задаёт рекомендации, а Write/NoCache дополнительно вызывает `msync(MS_SYNC)`. Эти режимы не эквивалентны единому «холодному кешу».

Наблюдается большой рост времени read/lseek с `O_DIRECT`; он включает особенности реализации мелких прямых операций и не изолирует только эффект удержания кеша. `0,00` у mmap означает округление, а не нулевую длительность. Два прогретых запуска всё же имеют большие ошибки страниц (1 и 3); прогрев не гарантирует их полного отсутствия. Baseline с N=1 предназначен для знакомства и проверки работоспособности; он не используется для оценки дисперсии или выбора N.

4. План, пилот и выбор числа измерений

Выполнена подтверждающая серия по заранее сохранённому плану. **До пилота** зафиксированы гипотеза, цель точности, метрика, прогревы, обработка выбросов и порядок запуска. После отдельного пилота вычислено и зафиксировано N, затем собрана основная серия.

4.1. Заранее заданные правила

Статистическая единица — один запуск процесса с **ITER=20** полными обходами. Время процесса T_i переводится в $x_i = 1000\frac{T_i}{20}$ мс/обход. Это нормированное время процесса, а не 20 независимых инструментальных отсчётов. Для каждой конфигурации выполняются два отдельных прогревочных запуска, они сохраняются в журналах и не входят в CSV измерений. Выбор **WARMUP=2** сделан заранее для исключения начального запуска.

Перед процессом граф восстанавливается; между его 20 обходами восстановления нет. В Write значение каждой посещённой вершины увеличивается на единицу при каждом обходе, связи не меняются. Файловый кеш прогревается перед **каждым** процессом, включая измеряемые. Все корректные наблюдения сохраняются независимо от времени. Критерий 1,5×IQR применяется только для описания отклоняющихся точек.

Порядок четырёх конфигураций по раундам: A–B–C–D, B–C–D–A, C–D–A–B, D–A–B–C, затем повтор. Это уменьшает смешение эффекта режима с общим дрейфом. В 30 раундах каждая группа занимает каждую позицию 7 или 8 раз.

4.2. Отдельный пилот

Пилот содержит **10 процессов на группу**, всего 40 наблюдений. Его данные не добавляются к основной выборке. До пилота задана цель: полуширина 95% интервала среднего не более **5% пилотного среднего**. Для группы g: $E_g = 0.05\overline{x}_{p,g}$.

Группа	Среднее, мс/обход	s пилота, мс/обход	Цель E, мс/обход	n_0	n по t
A	99,773924	1,369894	4,988696	2	3
B	247,566585	4,749459	12,378329	2	3
C	1,660815	0,203505	0,083041	24	26
D	3,543730	0,300423	0,177187	12	14

Начальное приближение ограничено снизу двумя наблюдениями: $n_0 = \max\left(2, \left\lceil \left(1.96\frac{s_p}{E}\right)^2 \right\rceil\right)$. Затем найдено наименьшее целое $n \geq 2$, для которого $t_{0.975,n-1}\frac{s_p}{\sqrt{n}} \leq E$. Вычисления используют неокруглённые данные. По правилу, заданному до пилота, $N = \max(30, 3, 3, 26, 14) = 30$ для всех групп. Предварительный минимум 30 обеспечивает число точек для распределений и запас относительно короткого пилота.

4.3. Фиксация плана и проведение основной серии

Плановое N=30 зафиксировано **до** начала основной серии. По пилотным s ожидаемые полуширины при N=30 составили 0,511527; 1,773477; 0,075990; 0,112180 мс/обход для A–D — меньше соответствующих E. Остановка происходит после 30 раундов, без изменения N в зависимости от полученного эффекта.

Приложение	Назначение
`plan/frozen-before-pilot-ru.md`	Неизменённые гипотеза и правила до пилота
`plan/before-pilot-manifest.json`	Хеши плана, измерительных сценариев и исходного кода
`data/pilot/results.csv`	40 отдельных пилотных наблюдений
`plan/selected-n.json`	Пилотные оценки, уточнение t и выбранное N=30
`plan/execution-ledger.jsonl`	Порядок фиксации плана, завершения пилота, выбора N и основной серии
`data/formal/results.csv`	120 наблюдений основной серии и исходные журналы

Проверка полноты: все 120 измеряемых процессов завершились с кодом 0; в каждом журнале ровно 20 сообщений об успешной обработке 436 905 вершин. Получены 2400 основных обходов, отдельно сохранены 8 прогревочных процессов. Прогревы и пилот не входят в N=30. Все корректные основные точки оставлены.

4.4. Метрика и разделение измерения с мониторингом

Основная длительность измеряется заранее выбранным `time.monotonic_ns()`, то есть CLOCK_MONOTONIC, вокруг цепочки `/usr/bin/time → taskset → программа`. Интервал включает запуск служебных процессов, открытие/закрытие или отображение файла, выполнение обходов и вывод в журнал. Запуск Python, компиляция, генерация, восстановление и прогрев файла происходят вне этого интервала.

GNU time дополнительно даёт user/sys, переключения контекста и ошибки страниц. `strace` и `perf` не используются внутри основной серии. В каждом журнале сохранены MONOTONIC, REALTIME и MONOTONIC_RAW для контроля таймера. Ресурсный мониторинг не заменяет wall и не увеличивает число независимых наблюдений.

Проверка инструмента выполнена в отдельной диагностике. Количество обходов там равно одному, поэтому диагностические длительности не добавляются к выборке основной серии с ITER=20. Данные background также предназначены для отдельного опыта конкуренции за CPU.

5. Статистическая обработка основной серии

Для нормированных отсчётов x_i рассчитаны среднее, выборочное стандартное отклонение со знаменателем $N - 1$ и стандартная ошибка среднего:

$$\overline{x} = \frac{\sum_{i=1}^{N} x_i}{N}, \quad s = \sqrt{\frac{\sum_{i=1}^{N} (x_i - \overline{x})^2}{N - 1}}, \quad \mathrm{SE} = \frac{s}{\sqrt{N}}.$$

$$\text{ДИ}_{95\%} = \overline{x} \pm t_{0.975,N-1} \frac{s}{\sqrt{N}}.$$

При N=30 квантиль t равен 2,04523, число степеней свободы — 29. ДИ описывает неопределённость **среднего**, а не диапазон отдельных запусков. Его интерпретация предполагает достаточную независимость и устойчивость наблюдений; систематическая ошибка часов в него не входит.

Группа	N	Среднее, мс/обход	s, мс/обход	95% ДИ, мс/обход
A	30	100,555595	2,778765	99,517987; 101,593203
B	30	246,274694	5,294152	244,297825; 248,251563
C	30	1,624222	0,168739	1,561213; 1,687230
D	30	3,498498	0,359218	3,364364; 3,632632

Пример для A: SE=0,507331 мс/обход; полуширина равна 2,04523×0,507331=1,037608 мс/обход. Средние длительности процессов из 20 обходов: 2,011112; 4,925494; 0,032484; 0,069970 с для A–D.

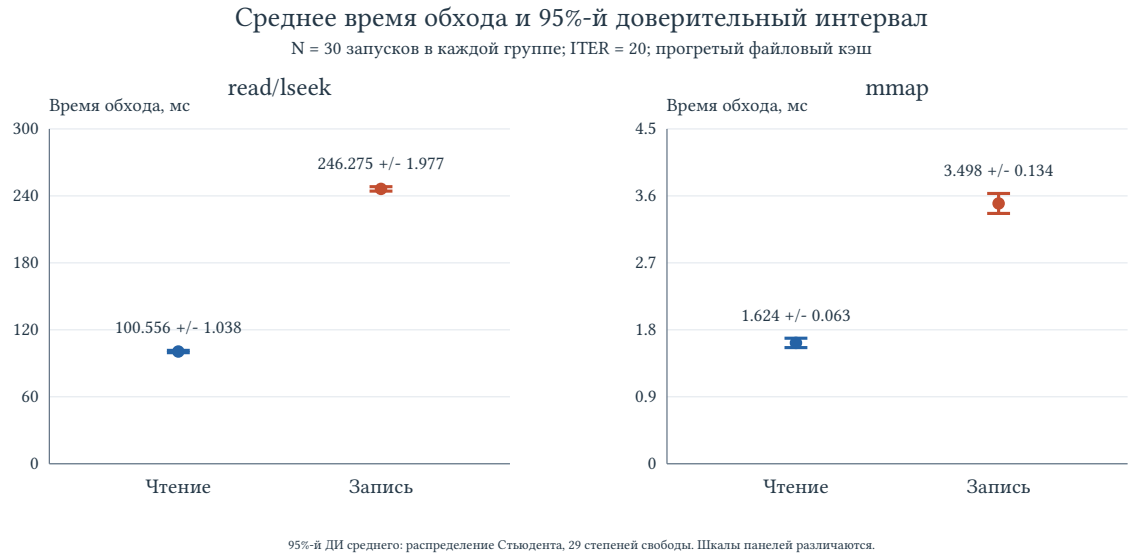


Рис. 1. Средние и 95% интервалы. В каждой панели шкала линейная и начинается с нуля; шкалы API различаются.

Группа	Медиана, мс/обход	IQR, мс/обход	Полуширина ДИ, % среднего
A	99,820700	2,708250	1,032%
B	246,059320	5,691328	0,803%
C	1,578831	0,125048	3,879%
D	3,437923	0,202383	3,834%

6. Распределения и проверка гипотезы



Рис. 2. Наложенные распределения Read/Write для каждого API. Ось X — мс/обход; Y — плотность. Цвета обозначают чтение и обновление.

Внутри каждой панели используются **общие границы столбцов** для двух режимов; площадь каждой гистограммы равна 1. У read/lseek ширина интервала около 2,428120 мс, у mmap — 0,205792 мс. Между API оси различаются. Столбцы отражают 30 нормированных процессных отсчётов в каждой группе

Распределение Write расположено правее Read в обеих панелях. У mmap видны отдельные более медленные процессы. Гистограмма показывает форму и разделение наблюдений, но не доказывает нормальность или независимость.

По каждому раунду r вычислено $d_r = x_{\mathrm{Write},r} - x_{\mathrm{Read},r}$. Для 30 пар использована та же формула t-интервала с выборочным отклонением **разностей**, а не сумма полуширин отдельных средних.

API	Пар	Средняя разность, мс/обход	s разностей, мс/обход	95% ДИ разности, мс/обход
read/lseek	30	145,719099	4,502238	144,037936; 147,400262
mmap	30	1,874277	0,347595	1,744483; 2,004071

Нижние границы обоих интервалов положительны. При принятой модели наблюдений данные поддерживают H_1 для каждого API: обновление занимает больше времени, чем чтение. Отношения средних Write/Read равны **2,449140** для read/lseek и **2,153954** для mmap, рост составляет 144,914% и 115,395% соответственно.

Цель 5% установлена для интервалов **средних отдельных групп**, а не для относительной точности разности. У mmap полуширина интервала разности составляет около 6,93% её среднего, это не противоречит заранее заданной цели. Для вывода о положительном направлении разности граница существенно выше нуля.

7. Объяснение через работу ОС

7.1. read/lseek и mmap

`lseek` задаёт позицию в открытом файле, `read` копирует данные из файлового пути ОС в пользовательский буфер. Программа читает запись вершины одним блоком размером 24 байта, затем извлекает её поля из буфера. В Write дополнительно выполняются позиционирование и `write` нового значения. Большое число обращений к ядру сохраняется даже при прогретом page cache.

`mmap` создаёт отображение файла в виртуальное адресное пространство. После отображения код читает поля через адреса памяти; при записи меняет общее отображение `MAP_SHARED`. Обычный доступ к вершине не вызывает отдельный `read` или `write`, но обращение к странице может потребовать обработчика ошибки страницы. Малые ошибки означают установление или обновление отображения без обязательного чтения данных с диска.

Отношение средних read/lseek к mmap составляет 61,9100 для чтения и 70,3944 для обновления. Это результат конкретных программ, графа и протокола; он не является универсальным преимуществом mmap. В коротких C/D относительная доля запуска, открытия и вывода заметнее. Cache не означает отсутствие всех обращений к накопителю: изменённые страницы могут записываться позднее, вне рассматриваемого интервала.

7.2. Ресурсные показатели основной серии

Средние приведены **на процесс из 20 обходов**, без нормирования на ITER. Дробные счётчики — результат усреднения целых значений.

Группа	user, с	sys, с	Малые ошибки	Большие	Добров. CS	Вынужд. CS
A	0,621667	1,232000	134,00	0	1,63	5,43
B	1,270000	3,287667	134,13	0	1,67	10,97
C	0,013667	0,005667	3334,23	0	1,63	0,13
D	0,021667	0,032667	54591,20	0	1,57	0,17

У B системное время примерно в 2,67 раза больше, чем у A. У mmap Write число малых ошибок значительно выше: каждое повторное отображение и доступ к изменяемым страницам создают дополнительную работу. Эти наблюдения согласуются с кодом; точный временной вклад каждого механизма отдельно не выделен.

Во всех основных процессах большие ошибки страниц равны нулю. Это совместимо с прогревом, но не доказывает полного отсутствия любого ввода-вывода. GNU time записывает user/sys с шагом 0,01 с; у коротких C/D их соотношения оценены грубо.

8. Устойчивость и контроль измерений

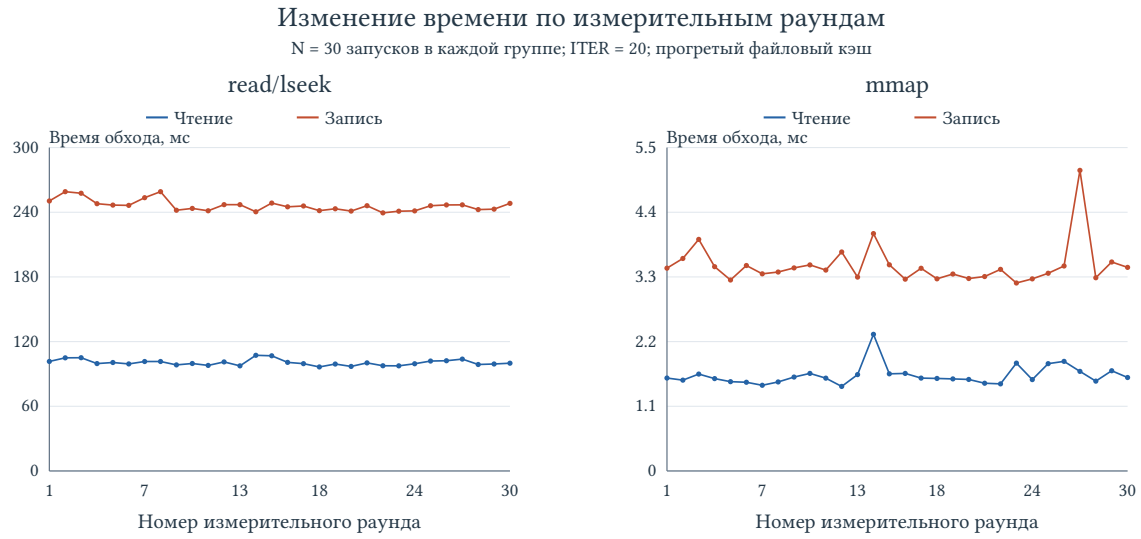


Рис. 3. Нормированное время по раундам. В каждой панели показаны обе конфигурации одного API; шкалы панелей различаются.

Среднее второй половины относительно первой изменилось на −1,919% (A), −1,955% (B), +1,114% (C), −1,572% (D). Чередование распределяет общий дрейф между режимами, но не доказывает независимость наблюдений. Интервалы трактуются как приближённые оценки случайной неопределённости данного протокола.

8.1. Достаточность заранее выбранного N

Фактические полуширины для A–D равны 1,037608; 1,976869; 0,063008; 0,134134 мс/обход. Они меньше заранее заданных E из пилота. Таким образом, N=30 достигло выбранной цели **по случайной неопределённости записанного среднего** и достаточно для вывода о направлении большого эффекта Read/Write при принятой статистической модели.

При сходном s увеличение N с 30 до 60 уменьшило бы полуширину примерно в 0,69 раза с учётом t-квантиля. Для уменьшения случайной полуширины примерно вдвое требуется около четырёхкратного N.

8.2. Контроль таймера

Заранее выбранный CLOCK_MONOTONIC сохранён в основном анализе. Все CSV длительности совпадают с соответствующими журналами. В 5 из 120 процессов REALTIME расходится с MONOTONIC более чем на 1 мс; календарное время не используется для вычисления длительностей. MONOTONIC превышает MONOTONIC_RAW по средним групп примерно на 7,75–7,80%; причина не установлена, независимая калибровка абсолютной шкалы не выполнена.

RAW используется только для проверки устойчивости **тех же запусков**, без замены метрики или пересчёта плана N. По RAW 95% интервалы Write–Read также положительны: 133,663455–136,659134 мс/обход для read/lseek и 1,618702–1,857198 для mmap. Направление эффекта сохраняется.

9. Диагностические инструменты

В отдельной диагностике получены **48 корректных процессов**: четыре конфигурации × четыре инструментальных режима × три повтора. Каждый процесс выполняет **один обход**. До измерений выполнен один прогревочный процесс Read из 20 обходов для каждого API; эти два процесса сохранены отдельно. Восстановление и прогрев файла происходят вне измерения. Инструменты внутри каждой конфигурации чередуются с циклическим сдвигом порядка.

Wall внешнего монотонного таймера включает `timeout`, `taskset` и выбранный инструмент. Ниже — средние и 95% t-интервалы в миллисекундах **на процесс с одним обходом**. При N=3 квантиль равен 4,30265.

Группа	Инструмент	N	Среднее, мс	95% ДИ, мс
A	Без инструмента	3	95,819	85,589; 106,050
A	time -v	3	97,763	92,151; 103,376
A	strace	3	4269,392	4001,118; 4537,665
A	perf	3	126,170	97,777; 154,564
B	Без инструмента	3	229,200	227,278; 231,122
B	time -v	3	236,564	219,612; 253,516
B	strace	3	8466,539	7938,620; 8994,458
B	perf	3	263,034	242,149; 283,918
C	Без инструмента	3	3,561	2,974; 4,148
C	time -v	3	4,246	2,460; 6,032
C	strace	3	5,370	3,282; 7,457
C	perf	3	26,668	−8,814; 62,149
D	Без инструмента	3	5,708	5,463; 5,954
D	time -v	3	6,222	5,022; 7,421
D	strace	3	7,267	6,977; 7,556
D	perf	3	17,191	−3,547; 37,929

Отношения средних strace / без инструмента: A — 44,5567; B — 36,9396; C — 1,5080; D — 1,2730. У A/B интервалы разделены: перехват большого числа системных вызовов резко меняет длительность. У C интервалы пересекаются, поэтому малая накладная стоимость точно не установлена. Интервалы perf у C/D широки, отрицательная нижняя граница симметричной t-аппроксимации при N=3 не означает отрицательное физическое время. Границы не обрезались.

Время под strace не используется как оценка обычного обхода. Для основной серии выбран лёгкий таймер с GNU time для счётчиков, а тяжёлые инструменты оставлены на этапе характеризации.

9.1. Системные вызовы и события perf

Для каждого режима три повтора strace дали одинаковые показанные ниже количества вызовов. Это счётчики **целого процесса с одним обходом**, включая загрузку программы и вывод в журнал.

Группа	read	write	lseek	Всего вызовов
A	436 907	2	436 906	873 850
B	436 907	436 907	873 811	1 747 660
C	1	2	0	40
D	1	2	0	40

У B добавлены примерно 436 905 позиционирований и записей. У C/D обход выполняется обращениями к памяти: записи в mmap не превращаются в вызов write на каждую вершину. Небольшие read/write у C/D относятся к работе процесса; счётчики не изолируют только доступ к графу. Значение времени системных вызовов в strace не заменяет внешнее wall.

В первом повторении time -v показал user/sys: A — 0,03/0,06 с, B — 0,04/0,18 с, C/D — 0,00/0,00 с с округлением. Max RSS равен 1440 КиБ для A/B и 11 680 КиБ для C/D; minor faults — 69, 70, 231 и 2790, major faults — 0. RSS не равен всему системному page cache. Это примеры одного диагностического обхода, а не средние основной серии из 20 обходов.

Использован отдельный бинарный файл `/usr/lib/linux-tools-5.15.0-194/perf`, версия 5.15.209: программные события реально получены, несмотря на отличие версии от ядра. Стандартная обёртка `/usr/bin/perf` не находит ожидаемую версию для WSL-ядра.

Группа	context-switches:u	cpu-migrations:u	page-faults:u	cache-misses:u
A	0 / 0 / 0	0 / 0 / 0	55 / 56 / 57	Не поддерж.
B	0 / 0 / 0	0 / 0 / 0	56 / 57 / 57	Не поддерж.
C	0 / 0 / 0	0 / 0 / 0	215 / 217 / 217	Не поддерж.
D	0 / 0 / 0	0 / 0 / 0	2777 / 2776 / 2775	Не поддерж.

В каждой ячейке показаны три повтора одного обхода. Суффикс `:u` — фактический пользовательский фильтр; эти показатели нельзя считать полной активностью ядра или всей системы. Нули — выданные значения с данным фильтром. `cache-misses` возвращает `<not supported>` и не подменяется нулём. Ошибка страницы не равна промаху CPU cache. Для переключений дополнительно анализируются счётчики GNU time.

9.2. Фоновая нагрузка

Проведён отдельный опыт для **обеих реализаций Read**: по три пары без нагрузки / с нагрузкой, ITER=20. Нагрузка — один Bash busy loop, закреплённый на том же CPU 2. Он запускается после восстановления и прогрева файла и завершается после измерения, порядок quiet/stress чередуется по повторам.

API	Условие	Среднее wall, с	s, с	Вынужд. CS
read/lseek	Без нагрузки	1,956309	0,013403	7,67
read/lseek	CPU-нагрузка	4,281070	0,039363	533,67
mmap	Без нагрузки	0,036017	0,005826	0,67
mmap	CPU-нагрузка	0,068207	0,007510	7,67

Среднее wall выросло в 2,1883 раза у read/lseek и в 1,8937 раза у mmap, вместе с числом вынужденных переключений. Это согласуется с разделением CPU между конкурирующими задачами. N=3 ограничивает точность коэффициента, особенно у короткого mmap, шумные процессы не смешиваются с основной серией.

10. Выводы и границы применимости

1. Для Seq, 10 МиБ, Cache обновление значений увеличило зарегистрированное среднее время обхода в **2,449140 раза у read/lseek** и **2,153954 раза у mmap**. Положительные интервалы парной разности поддерживают гипотезу для обеих реализаций, направление сохраняется при вспомогательном анализе RAW тех же процессов.
2. Гипотеза и правила обработки зафиксированы до пилота. По отдельному пилоту вычислено N=30, 120 основных наблюдений и 2400 обходов прошли проверки полноты. Пилот и заранее исключённые прогревы не добавлены к основной выборке, медленные корректные точки сохранены.
3. Фактические полуширины интервалов средних меньше заранее выбранных 5% пилотного среднего. Это достаточность для большого эффекта данного протокола по случайной неопределённости, а не обещание 5% абсолютной точности часов. Увеличение N сужает случайный интервал, но не устраняет систематические ограничения.
4. Структура системных вызовов и ресурсные показатели согласуются с дополнительной работой записи. mmap быстрее для этих программ за счёт доступа к отображению вместо множества отдельных вызовов на вершину. Результат не распространяется на произвольные размеры, топологии и режимы синхронизации.
5. Baseline охватывает случайный и последовательный графы и оба флага. Диагностика показывает сильное вмешательство strace в read/lseek, опыт с конкурирующей CPU-нагрузкой показывает рост длительности и переключений.